Reading PDFs in a Terminal, Fast, Locally and over SSH

The farleaf project

October 2026

Abstract

Terminals that speak the Kitty graphics protocol can show real images, which makes reading a PDF without leaving the terminal practical. Doing it well is mostly a matter of moving few bytes: a page is rendered once, sent in pieces, and afterwards only moved. This note describes how farleaf does that, how it picks the cheapest way to send an image, and how it stays inside the memory a terminal is willing to spend on images. Page turns take a few hundredths of a second locally and a few tenths over SSH.

1 Introduction

A PDF viewer in the terminal has an unusual constraint: everything it shows travels through a pseudo-terminal, possibly over a network, possibly through a multiplexer that keeps its own copy of the screen. A page rendered for a high-resolution display is easily 25 MB of pixels. Sent naively, as one image per page turn, it would take seconds over SSH and exhaust the image memory of some terminals after a handful of pages.

farleaf avoids most of that cost. It renders each page once, at exactly the size of the cells it will cover, sends it in pieces, and from then on asks the terminal to place images it already holds. The rest of this note explains the three ideas behind that: pieces, transports, and a memory budget.

1.1 What a reader needs

Reading is mostly scrolling and turning pages, with the occasional jump: to a section, a figure, a search hit. A viewer should make the common moves instant and the rare ones quick. It should also never show a blank where a page should be, which, as we will see, is the harder requirement.

1.2 The terminals

farleaf targets Ghostty and herdr, an agent multiplexer built on Ghostty's terminal core. Both accept images over the Kitty graphics protocol. They differ in how much image memory they keep: Ghostty about 320 MB per screen, a herdr pane only 64 MiB.

2 Design

2.1 Pages in pieces

A page is cut into horizontal strips, and, when zoomed wider than the screen, into tiles. A strip holds at most

$$B_{\max} = 4\,\mathrm{MB} \quad \text{and} \quad h_{\max} = \max\Big(2, \Big\lfloor \tfrac{R}{8} \Big\rfloor\Big) \ \text{rows}, \tag{1}$$

where R is the number of rows on screen. Small pieces keep every image under the terminals' limits (10 000 pixels on a side, 30 MiB once encoded), let the first strips show before the rest are rendered, and let a scroll move images instead of sending them again.

2.2 Choosing a transport

At startup farleaf asks the terminal one question with several parts: can it read shared memory on this machine, how big is a cell, and what is it. The answers decide how images travel (Table 1).

Where	Transport	Bytes per page
Ghostty, same machine	shared memory	a few names
Ghostty over SSH	PNG	0.3–1 MB
herdr, local or remote	zlib RGB	0.5–1 MB

Table 1: How images reach the terminal.

2.3 A memory budget

A terminal that runs out of image memory evicts images, and an evicted strip is a blank stripe on the page. farleaf therefore keeps its own account. If m_i is the memory piece i takes in the terminal, it keeps

$$\sum_{i \in \text{sent}} m_i \ \le \ M, \qquad M = \begin{cases} 192\,\mathrm{MiB} & \text{in Ghostty,} \\ 56\,\mathrm{MiB} & \text{in herdr,} \end{cases} \tag{2}$$

prefetching nearest-first until the budget is spent, and freeing the least wanted images before sending a new one.

3 Measurements

3.1 Setup

A fake terminal runs farleaf in a pseudo-terminal, emulates each terminal's limits, and checks after every key that the screen shows exactly what farleaf drew, pixel for pixel. The SSH link is modelled at 9 MB/s with 50 ms round trips and compression.

3.2 Results

Figure 1 shows the typical time from a key to a complete screen. Locally, a page turn mostly re-places images that are already there. Over SSH inside herdr, the time is dominated by herdr re-sending each new image to the outer terminal uncompressed.

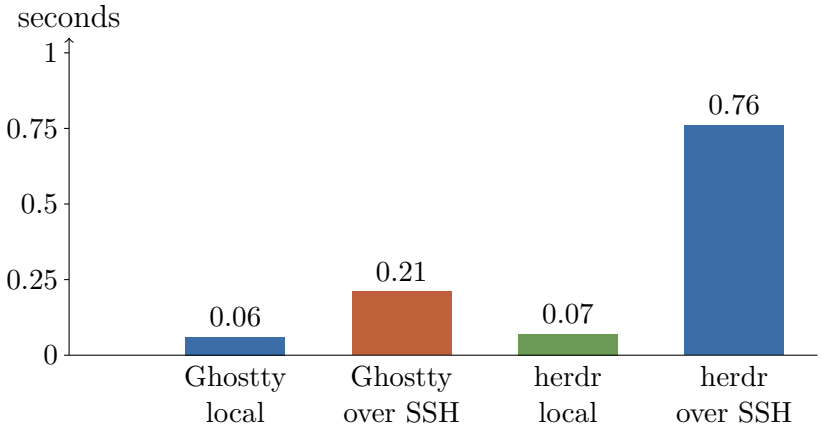


Figure 1: Typical seconds from a page turn to a complete screen.

4 Navigation

4.1 The outline

Most papers carry an outline: the sections and subsections, each pointing at a place on a page. farleaf lists it in a sidebar; moving through the list shows each place as you go, Enter stays there, and Escape returns to where you were.

4.2 Thumbnails

The other sidebar shows a small image of every page, as a desktop viewer does. Thumbnails are images like any other: they count against the same budget, are prefetched a few pages ahead, and are kept across zooms and resizes when their size does not change.

5 Implementation

5.1 Two threads

pdfium, the renderer, is not thread-safe, so a single worker thread owns it. The worker renders strips in the order the main thread asks for them: what is on screen first, then the rest of the page, then the next two pages and the previous one. When the reader moves on, the queue is replaced, and strips that are no longer wanted are never rendered.

A second thread owns the terminal. Small frames, which place images and draw the status line, jump ahead of queued uploads, so a key always gets an answer within one frame even while a large image is still on its way over a slow link. Uploads that stop being wanted while they wait are dropped before they are sent.

5.2 Reloading

Writing a paper usually means rebuilding the PDF many times. farleaf polls the file twice a second and reloads it once it has stopped changing, so a LaTeX run is read when it is done rather than halfway. The document is read into memory first, so a rebuild cannot change what is on screen underneath the reader. The reading position is kept: after a rebuild, the same paragraph is still on screen.

5.3 Search

Search runs on the worker too, page by page from the current one, and stops at the first page with a match. Matches are painted into the strips that contain them, like a highlighter pen: the paper turns yellow and the ink stays dark. Clearing the highlights re-renders only the strips that had any.

6 Limits

Some things are left out on purpose. Text cannot be selected, the mouse is not used, and tmux, which needs extra setup to pass images through, is not supported. Each of these would be useful, and each would add code that has to stay fast over a slow link. The guiding rule is that a feature has to earn its place by making reading better.

7 Conclusion

Fast terminal graphics are less about fast rendering than about sending little. Rendering at the size of the cells, cutting pages into pieces, asking the terminal what it can do, and respecting its memory make a PDF viewer that feels local even when the document is on the other side of an SSH connection.

A Keys

Keys	Action
`j k`	scroll
`l h`	next / previous page
`gg G`	first page / end
`z Z`	fit page / fit width
`/ n N`	search, next / previous match
`o`	outline
`t`	page thumbnails
`?`	every key
`q`	quit